

Reinforcement Learning SoSe 2025 - Task 1

Bericht

Tsvetan Stanchev

1) Einleitung

Im Rahmen dieser Projektarbeit wurde eine klassische Aufgabenstellung aus dem Bereich des Reinforcement Learnings bearbeitet: Ein Agent soll lernen, sich in einer diskreten, deterministischen Gridworld-Umgebung zurechtzufinden, um ein definiertes Ziel mit möglichst wenigen Aktionen zu erreichen. Hierbei kamen zwei etablierte RL-Algorithmen zum Einsatz – SARSA(λ) und Q-Learning(λ). Ziel war es, die vorgegebene Umgebung MazeWater2 aus der Datei gridworld.py erfolgreich zu lösen und den Lernverlauf mittels Plots nachvollziehbar zu dokumentieren. Darüber hinaus war es Teil der Aufgabe, eine eigene, kreativ gestaltete Umgebung zu entwerfen, welche klassische RL-Methoden vor neue Herausforderungen stellt. Um das Lernen effizienter zu gestalten und die Algorithmen optimal einzustellen, wurde das Hyperparameter-Tuning automatisiert mithilfe der Open-Source-Bibliothek Optuna durchgeführt. Der Bericht dokumentiert die angewandten Methoden, den Aufbau und die Eigenschaften der Umgebungen sowie die erreichten Ergebnisse.

2) Methoden und Implementierung

Zur Lösung der gestellten Aufgabe wurden zwei zentrale Reinforcement-Learning-Verfahren implementiert und verglichen: SARSA(λ) und Q-Learning(λ). Beide Methoden basieren auf tabellarischen Q-Werten und wurden um sogenannte *Eligibility Traces* erweitert, die es ermöglichen, nicht nur den unmittelbar nächsten, sondern auch vergangene Zustände im Lernprozess zu berücksichtigen. Die Implementierung erfolgte modular in Python, wobei sowohl das Training als auch die Auswertung vollständig automatisiert durchgeführt wurden.

Ein besonderer Fokus lag auf der Exploration, also der Frage, wie der Agent unbekannte Zustände erkundet. Hierzu wurden verschiedene Strategien getestet: Die klassische ϵ -greedy-Methode wählt mit Wahrscheinlichkeit ϵ eine zufällige Aktion, um neue Zustände kennenzulernen, und mit Wahrscheinlichkeit $1 - \epsilon$ die momentan beste bekannte Aktion. Um die Erkundung im Verlauf des Trainings schrittweise zu reduzieren, wurde zusätzlich eine Variante mit exponentiellem Zerfall von ϵ implementiert, bei der der Agent zu Beginn stark explorativ agiert und mit zunehmender Erfahrung immer deterministischer wird. Eine alternative Herangehensweise stellt die UCB1-Strategie (Upper Confidence Bound) dar, die nicht auf Zufall basiert, sondern systematisch die Unsicherheit im Q-Wert schätzt. Aktionen mit hohem Unsicherheitsmaß erhalten

hierbei eine höhere Priorität, was zu einer balancierten Exploration zwischen häufig und selten besuchten Zuständen führt.

Zusätzlich wurden zwei weitere Mechanismen integriert, um das Erkundungsverhalten zu unterstützen: Zum einen die optimistische Initialisierung der Q-Werte, bei der alle Aktionen anfangs mit hohen Werten versehen werden, sodass der Agent gezielt neue Aktionen ausprobiert. Zum anderen ein R-Max-Bonus, der einen Belohnungsaufschlag für selten besuchte Aktionen vergibt und somit ebenfalls die Diversität des Erkundungsverhaltens steigert.

3) Hyperparameteroptimierung

Ein zentraler Bestandteil dieser Arbeit war der Einsatz der Optimierungsbibliothek Optuna, um geeignete Hyperparameterkombinationen für die beiden Reinforcement-Learning-Algorithmen SARSA(λ) und Q-Learning(λ) zu finden. Da das Verhalten von RL-Agenten stark von den gewählten Hyperparametern abhängt, ist eine manuelle Abstimmung meist ineffizient und führt nicht zuverlässig zu optimalen Ergebnissen. Stattdessen wurde eine gezielte, automatisierte Suche mit Optuna durchgeführt, wobei Bayesian Optimization zum Einsatz kam. Dabei wurden folgende Parameter optimiert:

- γ (gamma) – Diskontierungsfaktor (Bereich: 0.90 bis 0.999): Dieser Parameter bestimmt, wie stark zukünftige Belohnungen im Vergleich zu unmittelbaren Belohnungen gewichtet werden. Ein hoher Wert ist in deterministischen Umgebungen sinnvoll, da das Ziel oft erst nach mehreren Schritten erreicht wird. Der Bereich beginnt bei 0.90, um zu starke Kurzfristigkeit zu vermeiden, und endet bei 0.999, was nahezu vollständige Zukunftsorientierung erlaubt.
- α (alpha) – Lernrate (Bereich: 0.0001 bis 0.5): Die Lernrate steuert, wie stark neue Informationen die bisherigen Q-Werte überschreiben. Ein zu hoher Wert kann zu instabilen Updates führen, während ein zu niedriger Wert den Lernprozess unnötig verlangsamt. Der gewählte Bereich erlaubt sowohl sehr vorsichtiges als auch schnelleres Lernen.
- ϵ (epsilon) – Explorationswahrscheinlichkeit (Bereich: 0.0001 bis 0.2): Dieser Parameter beeinflusst die Wahrscheinlichkeit, mit der der Agent eine zufällige Aktion auswählt (ϵ -greedy). Werte über 0.2 führen oft zu unnötigem Rauschen, während Werte unter 0.0001 kaum Exploration ermöglichen.
- λ (lambda) – Trace-Decay-Faktor (Bereich: 0.0 bis 1.0): λ bestimmt, wie stark vergangene Zustände in den Lernprozess über Eligibility Traces einfließen. Ein Wert nahe 0 entspricht einfachem TD(0)-Lernen, während 1 einer vollständigen Rückverteilung der Belohnung auf die Episode entspricht. Der gesamte Bereich wurde offengelassen, um optimale Strategien zu finden.
- Anzahl der Episoden (Bereich: 10 bis 10.000)

- exploration_strategy – Explorationsstrategie (nur „ucb1“ aktiviert): Zu Beginn wurden alle drei verfügbaren Strategien getestet – klassische ϵ -greedy, ϵ -greedy mit Zerfall sowie UCB1. Bereits nach kurzer Zeit zeigte sich jedoch, dass nur UCB1 in der Lage war, für die gegebene Aufgabe in ausreichend wenigen Schritten eine optimale Policy zu finden. Die anderen Strategien scheiterten entweder am zu langsamen Lernen oder an dauerhaft fehlerhaften Policies. Daher wurden in den weiteren Experimenten ausschließlich Tests mit UCB1 durchgeführt. Diese Strategie kombiniert Exploration und Exploitation besonders elegant, da sie die Unsicherheiten in der Q-Tabelle gezielt berücksichtigt. Im Gegensatz zu ϵ -greedy erfordert sie keinen manuell gewählten Zufallsfaktor, sondern balanciert das Erkundungsverhalten automatisch und adaptiv basierend auf dem bisherigen Wissen des Agenten.

- optimistic_init – optimistische Initialisierung (Bereich: 0.0 bis 5.0): Durch das Starten mit hohen Q-Werten werden alle Aktionen zunächst als attraktiv angesehen, was die Exploration fördert. Werte zu hoch über 5 führen allerdings zu künstlichem Verhalten, während 0 einer neutralen Initialisierung entspricht.

- r_max_bonus – R-Max-Bonus für selten besuchte Zustände (Bereich: 0.0 bis 5.0): Dieser Parameter fügt eine abnehmende Belohnung für selten gewählte Zustände hinzu, was deren Erkundung fördert. Bei 0 wird kein Bonus verwendet, bei höheren Werten wird Exploration aggressiver belohnt. Der Bereich wurde so gewählt, dass eine breite Exploration möglich ist, ohne das Lernziel zu verzerren.

- c_ucb – Explorationskoeffizient für UCB1 (Bereich: 0.1 bis 5.0): Der Parameter c steuert die Stärke des Unsicherheitsanteils bei der Aktionswahl. Niedrige Werte führen zu schnellerem Exploitieren, hohe Werte zu intensiver Exploration.

Alle Parameter wurden bei jeder Optuna-Trial neu kombiniert, ein vollständiges Training durchgeführt und anschließend mithilfe einer Bewertungsfunktion beurteilt, die sowohl Schritte zum Ziel als auch die benötigte Episodenzahl berücksichtigt.

Für jede getestete Hyperparameterkonfiguration wurde ein vollständiger Trainingslauf durchgeführt und anschließend mithilfe einer eigenen Evaluierungsfunktion bewertet. Diese überprüft in einer einzigen Episode, ob der Agent – basierend auf seiner final gelernten Policy – das Ziel zuverlässig erreichen kann. Dabei agiert der Agent deterministisch, also ohne Exploration, und wählt in jedem Schritt die aus seiner Sicht beste bekannte Aktion. Fällt der Agent ins Wasser oder gerät in eine Endlosschleife, wird der Versuch abgebrochen und die Anzahl der bis dahin benötigten Schritte als Maß für die Teilqualität der Policy zurückgegeben. Dadurch lässt sich erkennen, wie weit der Agent mit seiner Strategie kommt, selbst wenn das Ziel letztlich nicht erreicht wird – eine Policy, die erst nach 30 Schritten scheitert, ist deutlich besser als eine, die schon nach zwei fehlerhaft handelt.

Erreicht der Agent das Ziel erfolgreich, wird ein fixer Wert von 100 zurückgegeben. Dieser Wert fließt in die Berechnung eines Gesamt-Scores ein, der nach folgender Formel definiert ist: $score = steps * 100000 - episodes$. Das Produkt aus steps und dem Gewichtungsfaktor priorisiert Strategien, die das Ziel korrekt erreichen ($\rightarrow 100 * 100000$

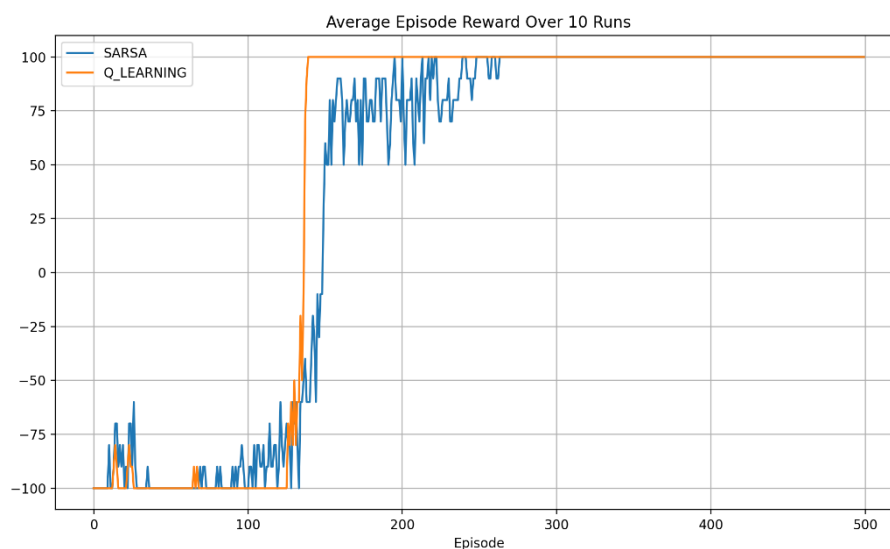
= 10.000.000 Punkte). Gleichzeitig wird durch den Abzug der Trainings-Episoden eine Lernzeitstrafe eingeführt, sodass nicht nur die Zielerreichung zählt, sondern auch wie effizient das Verhalten erlernt wurde. Diese Balance zwischen Performance und Trainingsdauer erlaubt es, nicht nur funktionierende, sondern auch schnell konvergierende Konfigurationen zu bevorzugen. Um die Robustheit der Bewertung sicherzustellen, wurden alle Trainingsläufe mehrfach wiederholt und die Ergebnisse gemittelt.

Die gefundenen Parameter wurden in einer Excel-Datei gespeichert und anschließend für die finale Evaluation verwendet.

4) Ergebnisse

Nach einer sechsstündigen Hyperparameteroptimierung mit Optuna konnte beobachtet werden, dass der Agent das Ziel in der Umgebung MazeWater2 in nur wenigen Schritten zuverlässig erreichen kann. Der besten Trial konnte das optimale Policy in nur 140 Schritten erreichen. Um die Qualität und das Verhalten dieser Trial genauer zu analysieren, wurde der speziell entwickelte Evaluator aus der Datei evaluation.py eingesetzt. Dieses Evaluationsskript ermöglicht es, das Algorithmus mit den besten gefundenen Parametern erneut auszuführen, dabei die durchschnittlichen Belohnungen über mehrere Wiederholungsläufe (10) zu visualisieren und zusätzlich die finale V-Tabelle sowie die Policy zu plotten. So lässt sich nachvollziehen, wie stabil und effizient das Verhalten des Agenten in der Trainingsumgebung tatsächlich ist. Auch potenzielle Schwächen in bestimmten Bereichen der Umgebung lassen sich durch die Darstellung der V-Werte gut identifizieren.

Das Diagramm zeigt den Belohnungsverlauf für den besten Trial. Beide Algorithmen starten mit negativen Werten, verbessern sich aber deutlich im Verlauf. Q-Learning erreicht bereits nach etwa 140 Episoden stabil die maximale Belohnung, während SARSA etwas langsamer und unregelmäßiger lernt, aber ebenfalls ein hohes Leistungsniveau erreicht.



5) Eigene Umgebung

Der letzte Teil der Aufgabe bestand darin, eine eigene, kreative Umgebung zu entwickeln, die auf der bestehenden Gridworld-Architektur basiert, aber für klassische Reinforcement-Learning-Algorithmen eine besondere Herausforderung darstellt. Mein Ansatz war es, eine Umgebung zu erstellen, in der ein zufällig generierter Labyrinth mit Verzweigungen und genau einem gültigen Lösungsweg erzeugt wird. Dafür implementierte ich eine neue Klasse namens MazeDFS, die mithilfe eines rekursiven Depth-First-Search-Algorithmus (DFS) ein perfektes Labyrinth erstellt – also eines, bei dem es keine Zyklen und nur einen eindeutigen Pfad vom Start zum Ziel gibt. Die Start- und Zielpositionen sind dabei fest definiert, während die Struktur des Labyrinths bei jedem Durchlauf neu generiert werden kann (optional reproduzierbar über einen Seed).

Der erste Test erfolgte mit einem Labyrinth der Größe 11, und die Ergebnisse zeigten, dass das Programm in der Lage ist, nach etwa sieben Stunden Optimierung mit Optuna eine gute Lösung für dieses Problem zu finden. Unten sind Beispiele dafür zu sehen – ein Diagramm mit dem Verlauf der kumulativen Belohnung sowie die gefundene optimale Policy in Form der Werttabelle.

